Escuela Politécnica Superior
Grado en Ingeniería
Informática

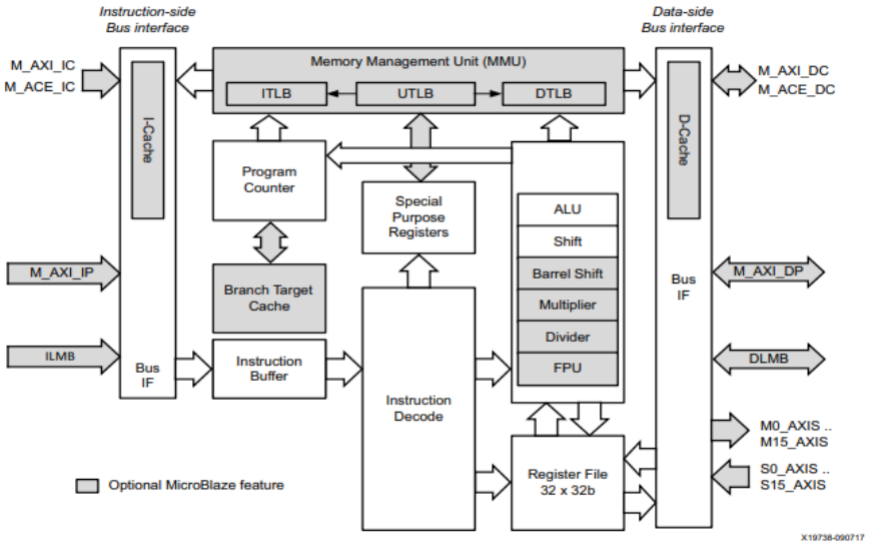
Práctica 4

Fernando Pérez Ballesteros
Pablo Rayón Zapater
Jorge Callado Araunabeña

UNIVERSIDAD
NEBRIJA

1. Introducción

Para esta práctica 4, se pide implementar la funcionalidad del microcontrolador Microblaze, mostrada en la imagen anterior, mediante un diseño VHDL.



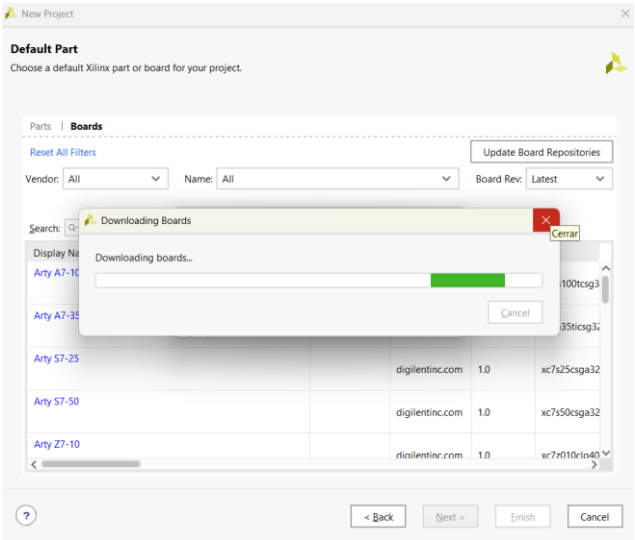
2. Pasos que debemos seguir.

Debido a el gran tamaño de la herramienta Vitis y siguiendo las instrucciones del profesor en la realización de la práctica, no tenemos las capacidades de instalar dicha herramienta, por lo tanto, empezaremos con creación del proyecto.

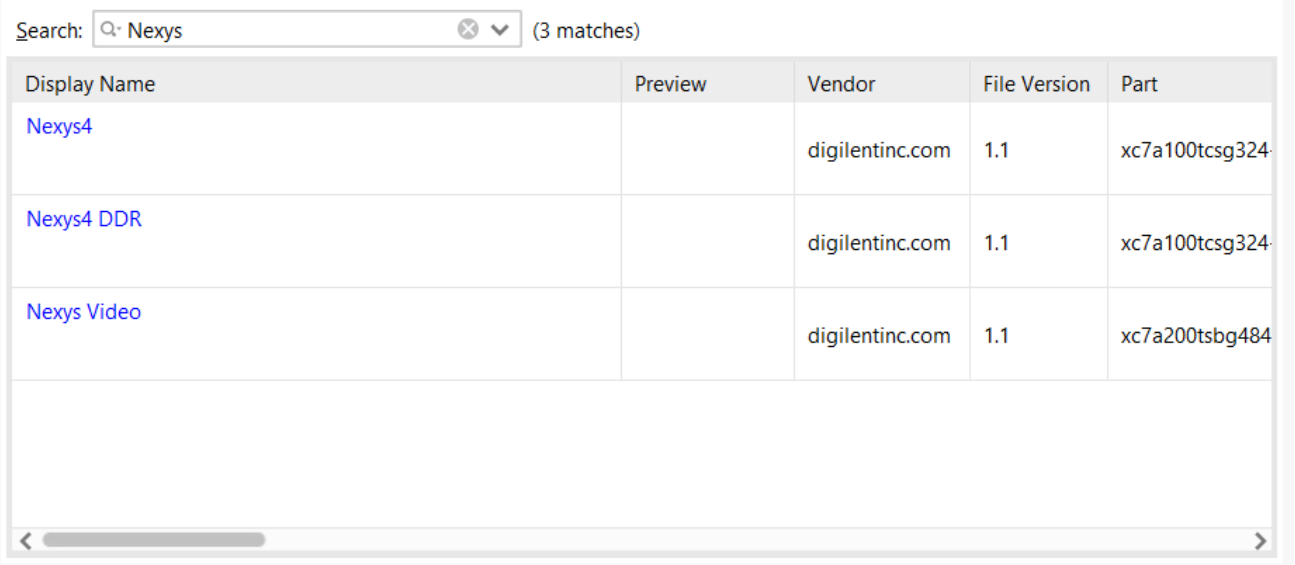
3. Creación del proyecto.

En caso de utilizar una FPGA, lo habitual es tener un diseño digital implementado previamente y luego proporcionar una interfaz para la comunicación con el procesador. En la práctica actual, solo se van a utilizar bloques IP suministrados por Xilinx y no se necesita agregar ningún archivo de diseño adicional. Para comenzar, se debe crear un bloque de diseño en un nuevo proyecto en Vivado, seleccionando el objetivo de Nexys 4 DDR y descargando los componentes necesarios.

Ya que el target de Nexys no nos aparecía, tuvimos que actualizar los repositorios.



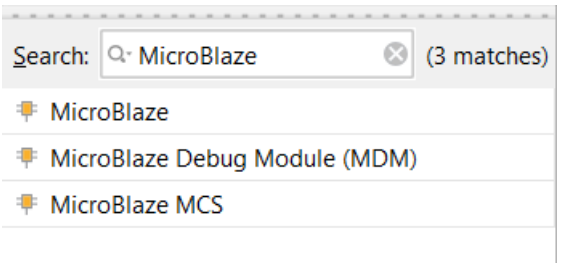
Una vez actualizado, nos aparecerán distintos targets y deberemos elegir el Nexys4 DDR.



Después de haber generado el proyecto, debemos crear un diagrama de bloques en el menú lateral. Para hacerlo, hemos ido a Flow Navigator, seleccionamos IP Integrator y luego elegimos Create Block Design. Al crear el diagrama, se pedirá que se proporcione un nombre sin espacios. En este caso, el nombre elegido fue "microblaze_nexys".

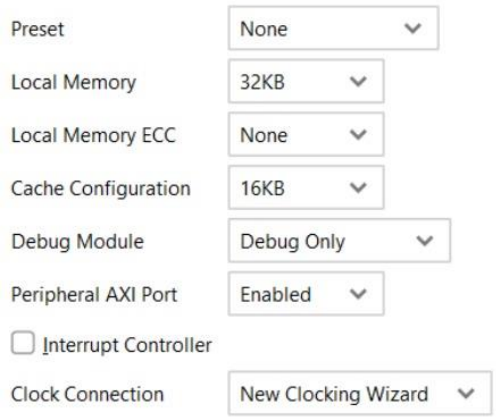
4. Microblaze.

El diagrama de bloques es una herramienta que nos permite crear diseños complejos utilizando múltiples componentes. En este caso, el diseño que se va a crear estará basado en el microprocesador MicroBlaze y se utilizarán únicamente componentes proporcionados por Xilinx en forma de IPs. Para añadir la IP, dberemos hacer click en “add IP” y buscaremos “Microblaze” parta posteriormente seleccionarla.

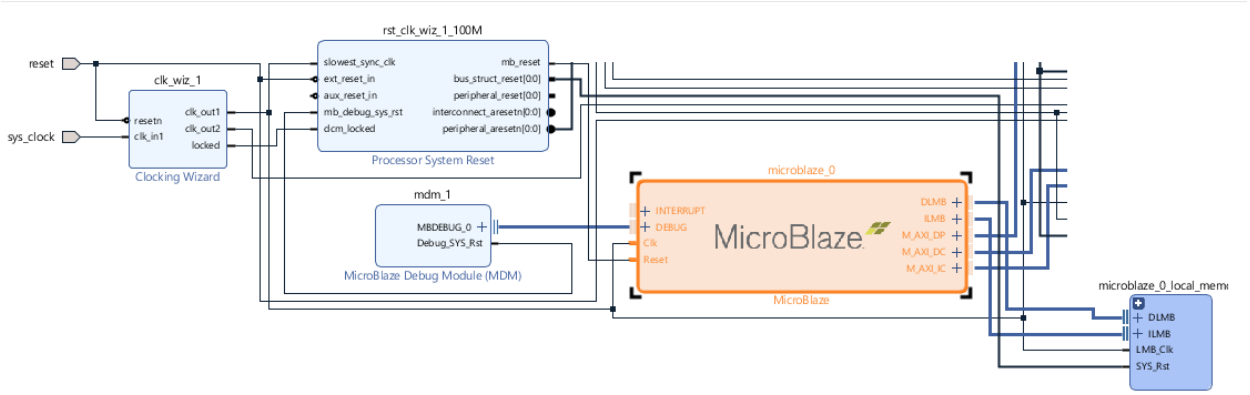


Tras seleccionarlo, podremos verlo en nuestro diagrama.

Una vez nos aparezca este procesador, deberemos configurarlo. Para ello, clicaremos en “Run Block Automation” y lo configuraremos con los siguientes parámetros.



Lo que hemos hecho en este caso es utilizar las herramientas de Vivado para que se incluyan automáticamente las IPs necesarias para el correcto funcionamiento del microprocesador, tales como un reloj, una memoria interna, un módulo de reset para el procesador y memoria, y un módulo que maneja el debug. Además, también se han establecido las conexiones necesarias entre estos bloques para garantizar el correcto funcionamiento del diseño. En resumen, se ha realizado una configuración automática de los bloques necesarios para el microprocesador y se han establecido las conexiones correspondientes.

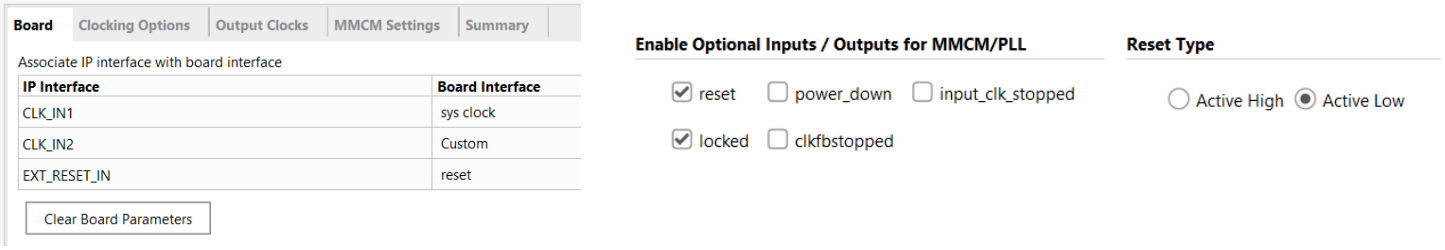


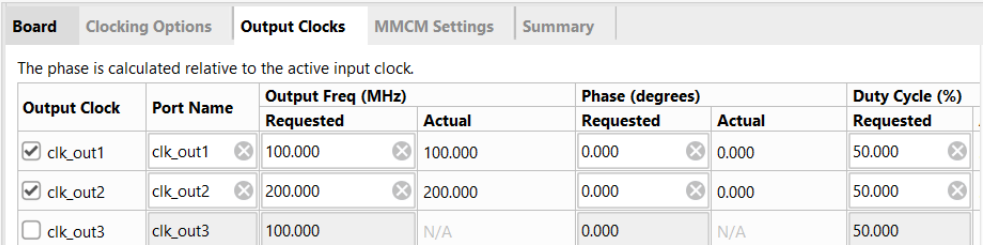
Estos son los bloques que se generan tras dicha configuración.

5. Modificar clk_wiz_1

La modificación de la configuración del bloque reloj se realiza para adaptarlo a las necesidades específicas del diseño. En este caso, se quiere incluir una memoria RAM que funcione a 200 MHz, mientras que el procesador trabaja a 100 MHz. Por lo tanto, se debe modificar la configuración del bloque reloj para que sea capaz de generar una señal de reloj a 200 MHz.

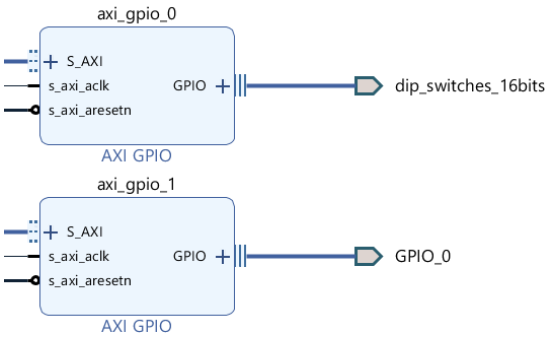
Para realizar esta modificación, debemos hacer doble clic en el bloque de reloj (clk_wiz_1) y se abrirá la ventana de configuración correspondiente. En la pestaña "Board", debemos seleccionar el reloj y reset de entrada que provienen de la FPGA (sys clock y reset). Además, en la pestaña “Output Clocks” debemos realizar otros cambios necesarios para generar la señal de reloj a la frecuencia deseada, como por ejemplo, seleccionar una frecuencia de salida de 200 MHz y configurar el reset como “active low”. De esta forma, el bloque de reloj estará configurado correctamente para generar la señal de reloj necesaria para la memoria RAM.



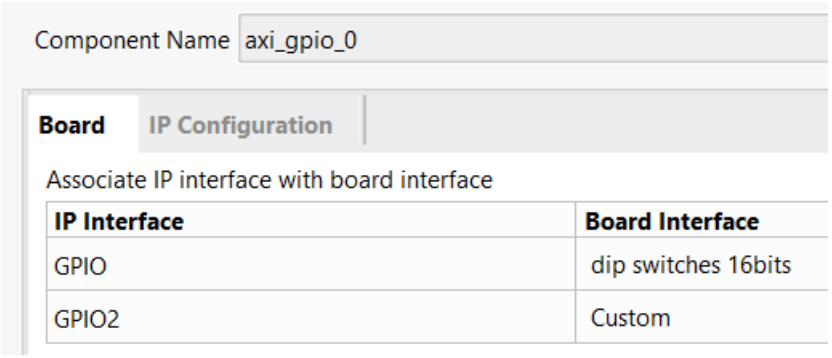


6. Uso de GPIO.

En este paso, nos vamos a centrar en dos grupos: los interruptores y los led. Ambos en la carpeta de GPIO (General Purpose Input/Output). Por lo tanto, añadiremos dos bloques GPIO para manejar los componentes anteriores.



Debemos configurar previamente axi_gpio_0 para poder conectar su salida, al led de la placa.

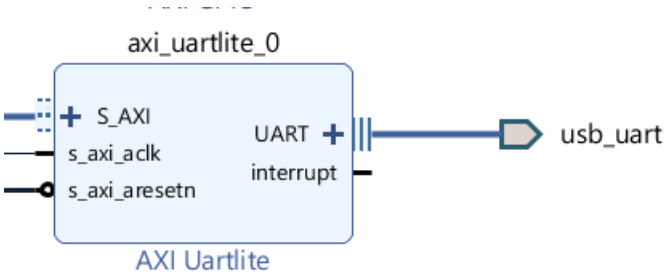


Por otra parte debemos indicarle a “axi_gpio_1” que la salida es externa mediante GPIO(“GPIO_0”).

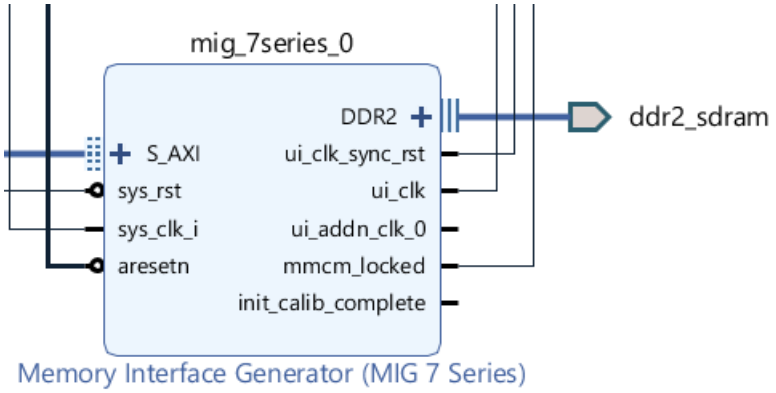
7. UART.

El siguiente paso consiste en agregar una interfaz UART (Universal Asynchronous Receiver-Transmitter) al diseño. Esta interfaz nos permitirá cargar el programa software y monitorear su funcionamiento desde un terminal mediante una conexión USB. Para ello, se va a utilizar un bloque predefinido de Vivado llamado "AXI Uartlite".

Una vez que hemos agregado el bloque de la interfaz UART al diseño, tenemos que realizar la conexión adecuada entre los diferentes bloques del diseño para que puedan comunicarse entre sí. Para realizar esta tarea de forma automática, tenemos que clicar en el botón "Run Connection Automation" como hicimos previamente en el Microblazer. De esta manera, Vivado se encargará de realizar las conexiones necesarias de forma automática, tanto para el nuevo bloque de la interfaz UART como para los bloques que ya habíamos agregado anteriormente.

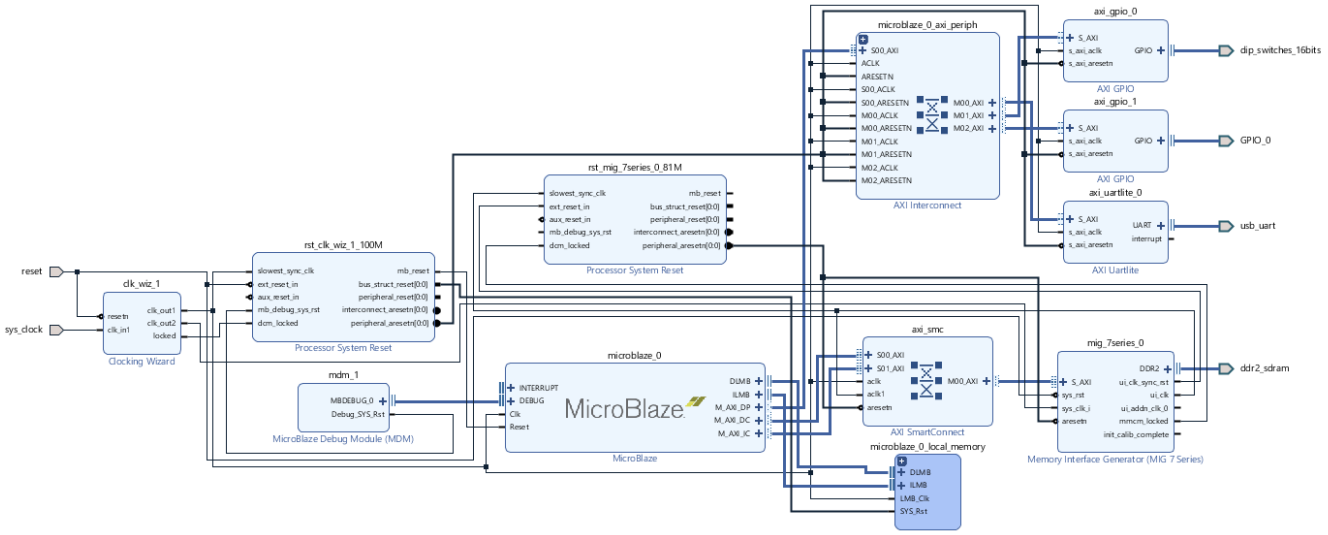


En este punto, vamos a agregar una nueva memoria a nuestro proyecto, que va a ser diferente a la memoria local del microprocesador. Para ello, vamos a utilizar la memoria RAM DDR2 que está presente en la placa Nexys. Sin embargo, aún necesitamos un controlador para poder trabajar con ella. Para esto, usaremos el generador de interfaz de memoria (MIG) que nos proporciona Vivado, ya que implementar un controlador de memoria RAM desde cero no es una tarea sencilla. Una vez hayamos añadido dicho componente, deberemos hacer como los anteriores componentes para generar las conexiones necesarias mediante "Run Connection Automation".



Si nos fijamos en el diseño, en la conexión “sys_clk_i” del bloque “mig_7series_0”, está conectada con el reloj generado de 100 MHz “clk_out1”. Sin embargo, la memoria RAM funcionaba a 200 MHz por lo que modificamos esta conexión para su correcto funcionamiento.

Por último, también tenemos que indicar que las conexiones con la memoria RAM son externas. En concreto son conexiones a la memoria integrada en la FPGA. En el menú izquierdo, en la pestaña Board, buscamos la interfaz “DDR2 SDRAM”. Arrastramos esta interfaz hasta el controlador “mig_7_series” se nos hará la conexión externa de forma automática. Y con esto quedaría finalizada nuestro diagramas de bloques.



8. Generación de bitstream.

Como último paso, para poder desarrollar el software que queramos,debemos generar antes el bitstream, para ello debemos abrir en el menú superior la ventana “File” -> “Export” -> “Export Hardware” y seleccionamos la opcion de incluir bitstream. Debido a la incapacidad de la instalación de Vitis, como hemos comentado anteriormente, el profesor de prácticas nos facilitó la visualización del funcionamiento con ello instalado desde su ordenador.

9. Explicación de componentes utilizados en la práctica.

Microblaze: es un microprocesador softcore desarrollado por Xilinx, diseñado para ser implementado en una FPGA. Es completamente personalizable y se integra con otros componentes del sistema en un solo chip FPGA, lo que permite una mayor flexibilidad en el diseño de sistemas embebidos. Además, es fácil de usar y se puede utilizar para diseñar controladores de dispositivos, sistemas de procesamiento de señales, sistemas de comunicación y sistemas de control de robots.

SoC: Son las siglas de System-on-Chip, que se refiere a un circuito integrado que combina varios componentes de hardware en un solo chip, como un microprocesador, memoria, interfaces de entrada/salida y otros periféricos.

GPIO: Es una abreviatura de General Purpose Input/Output (Entrada/Salida de Propósito General), se refiere a pines que se pueden configurar para recibir o enviar señales de distintos tipos.

UART: Es un acrónimo de Universal Asynchronous Receiver/Transmitter (Receptor/Transmisor Asíncrono Universal), es un dispositivo que se encarga de transmitir y recibir datos de forma serial en un canal de comunicación.

10. Ejemplo de otra funcionalidad similar en esta FPGA.

Para implementar un controlador de pantalla gráfica utilizando la interfaz HDMI de la placa Nexys, se podría utilizar un bloque predefinido de Vitis o Vivado, como por ejemplo el IP "Video Timing Controller", que se encarga de generar señales de sincronización y timing para la pantalla.

Una vez que se tenga el controlador de pantalla, se puede programar en software la generación de imágenes para mostrar en la pantalla. Esto se podría hacer utilizando lenguajes de programación como C o C++, y bibliotecas gráficas como OpenGL.

Además, la placa Nexys incluye varios puertos GPIO que se podrían utilizar para interactuar con el usuario. Estos puertos podrían utilizarse, por ejemplo, para controlar la pantalla táctil que también está integrada en la placa, permitiendo al usuario interactuar con la pantalla mediante gestos táctiles.

En resumen, se podría implementar un sistema completo de visualización y control de usuario utilizando la pantalla y los puertos GPIO de la placa Nexys, lo que abriría la posibilidad de crear una amplia gama de aplicaciones, desde sistemas de control de máquinas hasta interfaces de usuario para sistemas de entretenimiento.